

Iterative programming

Iterative programming

The concept of iteration underlies any operation in programming languages. In R many operations are vectorized. This makes the language more efficient and cleaner BUT hides the concept of iteration.

For example, the `sum()` function lets us add up a vector of numbers.

```
1 # add up the numbers from 1 to 10  
2 sum(1:10)
```

```
[1] 55
```

But what is hidden underneath?

Example: if I ask you to use the `print()` function to write “hello world” in the console 5 times, how would you do it?

```
1 msg = "hello world"  
2 print(msg)
```

[1] "hello world"

```
1 print(msg)
```

[1] "hello world"

```
1 print(msg)
```

[1] "hello world"

```
1 print(msg)
```

[1] "hello world"

```
1 print(msg)
```

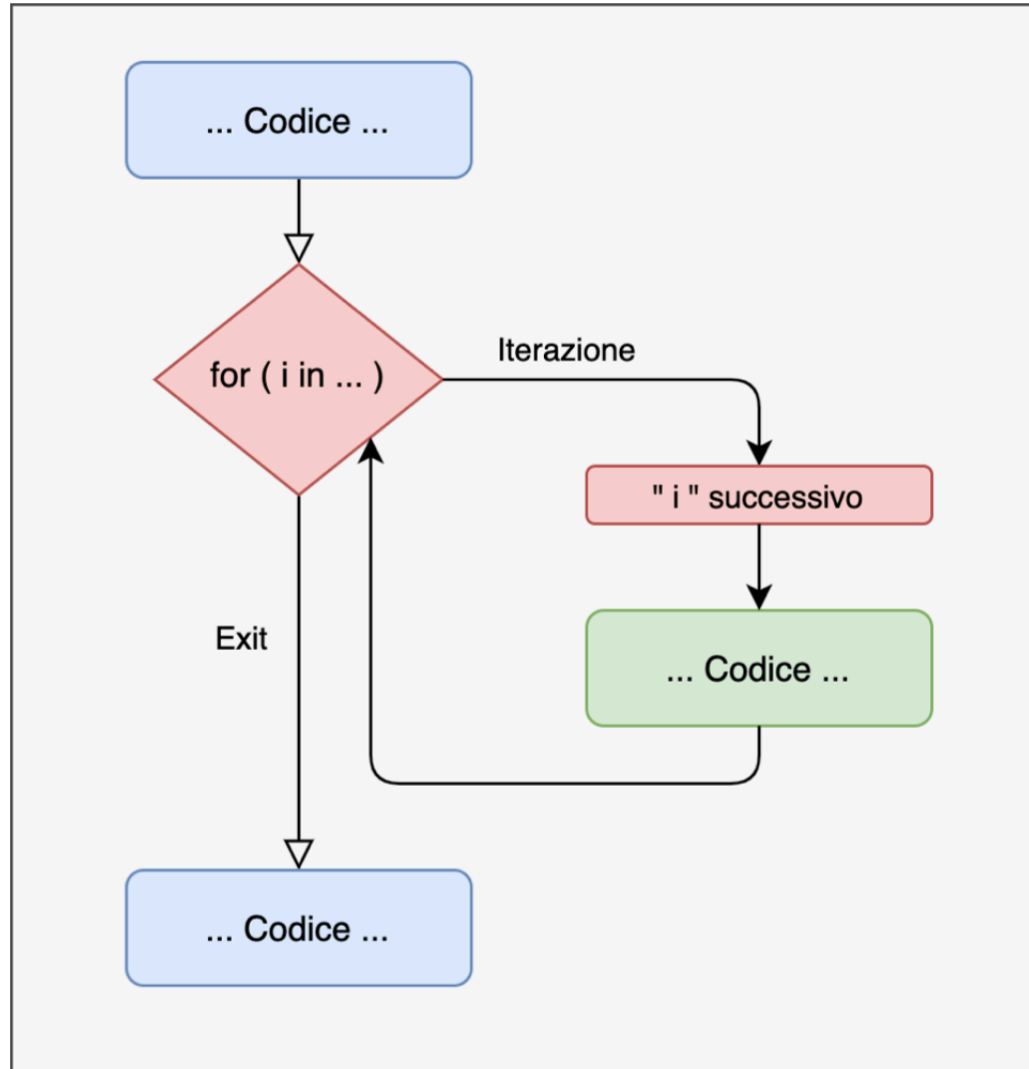
[1] "hello world"

What we are missing (and what the `sum()` function does) is a way to repeat a given operation without actually repeating the code manually.

There are several constructs that let us repeat operations; the most used are:

- The `for` loop
- The `while` loop
- The `repeat` loop
- The `*apply` family

The **for** loop



The **for** loop is a structure that lets us repeat a portion of code a finite, pre-determined number of times.

A for loop is written as:

```
1 n = 10
2 for(i in 1:n){
3   # which operations
4 }
```

If I want to print something 5 times, I can simply use a for loop:

```
1 for(i in 1:5){
2   print(paste("hello world", i, sep = "_"))
3 }
```

```
[1] "hello world_1"
[1] "hello world_2"
[1] "hello world_3"
[1] "hello world_4"
[1] "hello world_5"
```

Breaking down the **for** loop

- **for(){ }**: this is the implementation in R (similar to the if statement)
- **i**: this is called the iterator or index. It is a generic index that can take any value and name. By convention it is called i, j etc. It keeps track of the number of iterations our loop has to do
- **in <values>**: this indicates the values the iterator will take inside the loop
- **{ # operations }**: these are the operations the loop must perform

The power of the for loop lies in the fact that the **iterator** `i` takes the values of the vector specified after `in`, one at a time:

```
1 for(i in 1:10){  
2   print(i)  
3 }
```

```
[1] 1  
[1] 2  
[1] 3  
[1] 4  
[1] 5  
[1] 6  
[1] 7  
[1] 8  
[1] 9  
[1] 10
```

For with an index vs without

i represents the index of the elements of the vector `vec`

```
1 vec=1:5
2
3 for(i in 1:length(vec)){
4
5     print(vec[i])
6
7 }
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

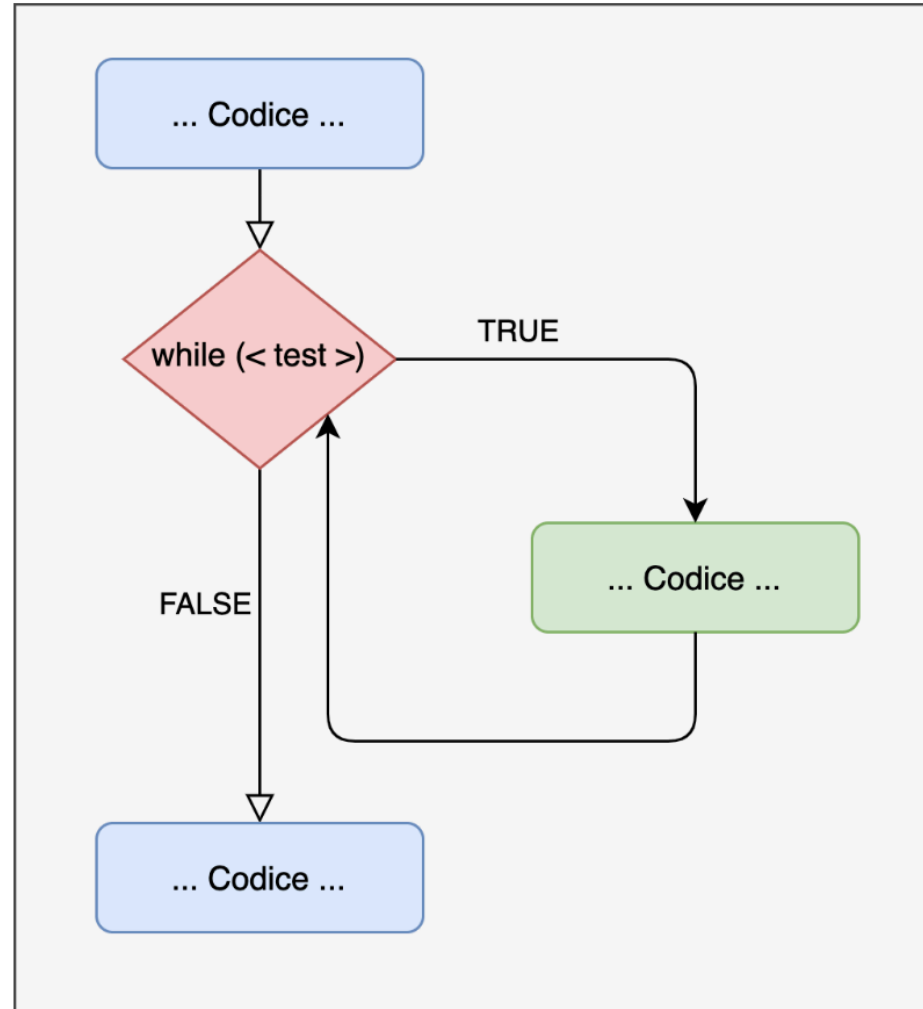
i directly represents each element of the vector `vec`

```
1 vec=1:5
2
3 for(i in vec){
4
5     print(i)
6
7 }
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

The **while** loop

The **while** loop uses a logical condition instead of an iterator and a range of values as in the for loop. The loop keeps going as long as the condition is **TRUE**:



Try writing this **while** loop, see what happens and understand why

```
1 x = 10
2
3 while (x < 15) {
4
5     print(x)
6
7 }
```

The **while** loop is not pre-determined, so it always needs a way to be stopped, by making the condition false.

```
1 x = 5
2
3 while (x < 15) {
4     print(x)
5     x = x + 1
6 }
```

```
[1] 5
[1] 6
[1] 7
[1] 8
[1] 9
[1] 10
[1] 11
[1] 12
[1] 13
[1] 14
```

The **repeat** loop

The logic of the **repeat** loop is very similar to that of the **while** loop, but with 3 important differences:

1. it always performs at least one iteration
2. it emphasizes repetition until a condition is met
3. it uses the **break** command to stop

The **repeat** loop

Roll the die until a 6 comes up:

```
1 repeat{  
2   die_roll = sample(1:6, 1)  
3   print(die_roll)  
4   if(die_roll == 6) break  
5 }
```

```
[1] 3
```

```
[1] 6
```

repeat vs. while

repeat evaluates the condition at the **end** of the iteration, while **while** does it at the **beginning**. If the condition is not **TRUE** at the start, while does not run, whereas repeat does:

```
1 i=1
2 repeat {
3   print(i)
4   i=i+1
5   if(i > 3)break
6 }
```

```
[1] 1
[1] 2
[1] 3
```

```
1 i=1
2 while (i < 4) {
3   print(i)
4   i=i+1
5 }
```

```
[1] 1
[1] 2
[1] 3
```

Example: the `sum()` function

Imagine we did not have the `sum()` function and wanted to recreate it: how would we do it? Any ideas?

Sum as iteration

Let's break the sum down conceptually; we add up the numbers from 1 to 10:

- I take the first one and add it to the second (total = $1 + 2$)
- I take the total and add it to the 3rd element: total = total + 3 ...

In practice we have:

- our vector to sum
- a total object that progressively accumulates the previous sums

Sum as iteration

```
1 total = 0 # initialize the total to 0
2 x = 1:10 # vector x
3
4 for(i in 1:length(x)){ # for the whole length of x = 1:10
5     total = total + x[i]
6 }
```

Let's put everything inside a function (i.e., I create a function that replicates what the sum function does)

```
1 my_sum = function(x){  
2  
3   total = 0 # initialize the total to 0  
4  
5   for(i in seq_along(x)){ # equivalent to 1:length(x)  
6  
7     total = total + x[i]  
8  
9   }  
10  
11   return(total)  
12  
13 }  
14  
15 x = rnorm(100, mean = 0, sd = 1)  
16  
17 my_sum(x)
```

```
[1] 0.4154756
```

```
1 sum(x)
```

```
[1] 0.4154756
```

Iteration, applications

The **for** loop is also useful, for example, to simulate data. Suppose we have a variable *y* (income) that depends on age:

1. I load the dataset I created:

```
1 mydf = read.csv(file = "data/mydf_2.csv") # load the dataset
2
3 str(mydf)
```

```
'data.frame':  30 obs. of  4 variables:
 $ id      : int  1 2 3 4 5 6 7 8 9 10 ...
 $ age     : int  47 50 23 39 28 15 17 38 19 42 ...
 $ age_cat: chr   "adult" "adult" "young" "adult" ...
 $ age_z   : num  1.394 1.665 -0.774 0.672 -0.322 ...
```

2. With a **for** loop, I simulate the variable **y** (income) assuming it depends on **age_z** (age):

```
1 set.seed(111) # for reproducibility
2
3 niter = 1000
4 res_all = rep(NA, niter)    # all estimated effects
5 p_val = rep(NA, niter)     # all p-values
6
7 # Observed data
8 age_z = mydf$age_z
9 eff_age = .2 # age effect
10 n_obs = length(age_z)
11
12 for(i in 1:niter) {
13   y = 0 + eff_age * age_z + rnorm(n_obs, 0, 1) # always add some noise
14
15   # linear model
16   mod = lm(y ~ 1 + age_z)
17
18   # extract the effect estimate
19   res_all[i] = coef(mod)[2]
20
21   # p-value of age_z
22   p_val[i] = summary(mod)$coefficients[2, 4]
23 }
```

Result:

```
1 # Keep only significant results
2 res_sig = res_all[p_val <= 0.05]
3
4 # Descriptive statistics
5 cat("Proportion significant = Power:", mean(p_val <= 0.05), "\n")
```

Proportion significant = Power: 0.165

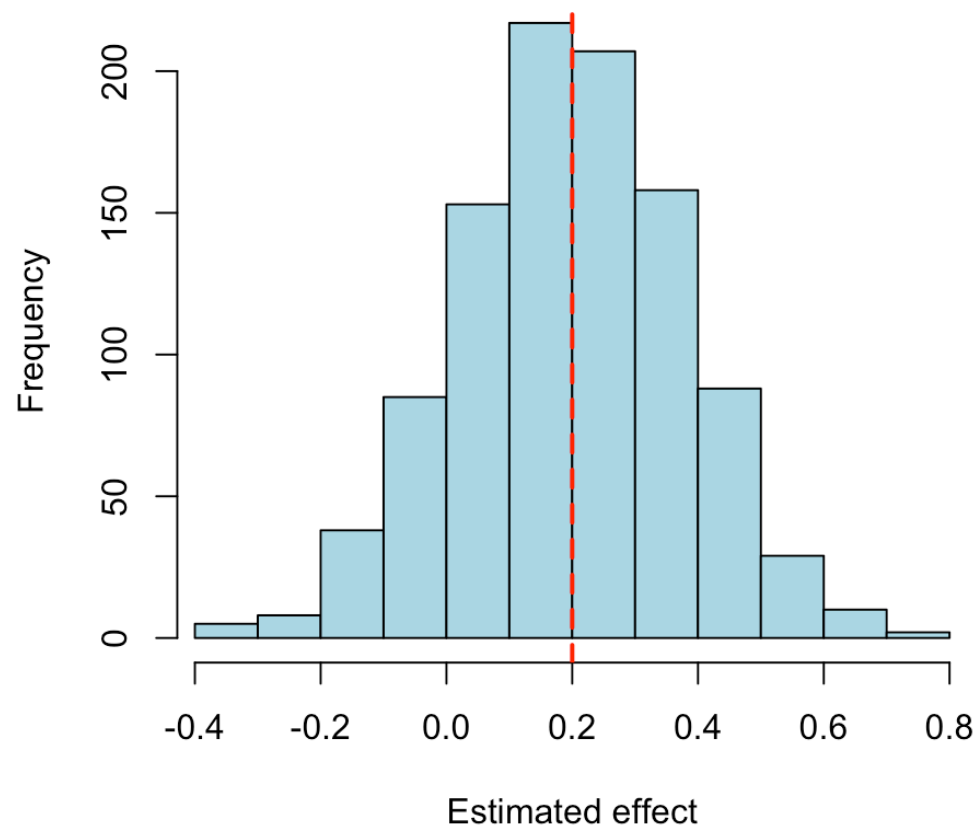
```
1 cat("Mean of all effects:", mean(res_all), "\n")
```

Mean of all effects: 0.1975066

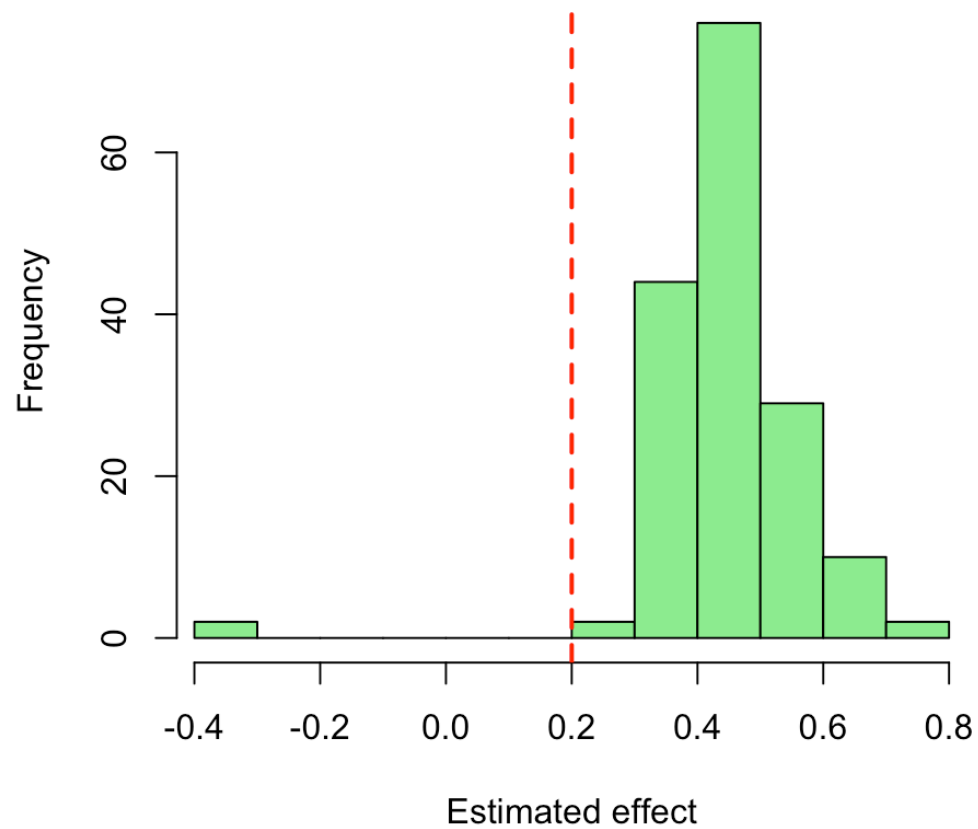
```
1 cat("Mean of significant effects:", mean(res_sig), "\n")
```

Mean of significant effects: 0.4470989

All results



Significant results only ($p \leq 0.05$)



It is precisely thanks to the for loop that we can run a ***power simulation***.

To learn more, have a look at this [page](#)! :)

Nested **for** loops

Once you understand the iterative structure, you can easily extend it by putting a loop inside another one:

```
1  for(i in 1:3){ # level 1
2
3    for(j in 1:3){ # level 2
4
5      print(paste(i, j))
6
7    }
8
9  }
```

```
[1] "1 1"
[1] "1 2"
[1] "1 3"
[1] "2 1"
[1] "2 2"
[1] "2 3"
[1] "3 1"
[1] "3 2"
[1] "3 3"
```

Imagine we have two experimental groups (for example, a Control group and a Treatment group), each with 10 participants. We want to measure a variable (for example, reaction time) for each participant and record the results in a dataframe:

```
1 # Define the study parameters
2 groups = c("Control", "Treatment") # Group names
3 n_subj=10 # Number of subjects per group
4 tot_subj = n_subj * length(groups)
5
6 # Create a data.frame to store the data
7 res=data.frame(
8
9   # Group factor
10  Group = factor(x = rep(groups, each = n_subj)),
11
12  # Subject factor
13  Subject = factor(x = 1:tot_subj),
14
15  # Reaction time: numeric vector with rt = 0
16  RT = numeric(tot_subj)
17 )
```

```
1 head(res, 5) # look at the first observations of res
```

	Group	Subject	RT
1	Control	1	0
2	Control	2	0
3	Control	3	0
4	Control	4	0
5	Control	5	0

```
1 tail(res, 5) # look at the last observations of res
```

	Group	Subject	RT
16	Treatment	16	0
17	Treatment	17	0
18	Treatment	18	0
19	Treatment	19	0
20	Treatment	20	0

```
1 # Nested loop to simulate data collection
2 set.seed(42) # For reproducibility of the simulated data
3
4 k = 1 # reaction time iterator
5
6 # how many levels does Group have?
7 for (i in 1:length(levels(res$Group))) {
8
9   # start with the first group, which is the control group
10  group = levels(res$Group)[i]
11
12  for (j in 1:n_subj) { # n_subj for each group
13
14    # is it the control group?
15    RT = ifelse(group == "Control",
16               yes = rlnorm(1, mean = log(.500), sd = .050),
17               no = rlnorm(1, mean = log(.450), sd = .050))
18
19    res$RT[k] = RT
20    k = k + 1
21  }
22 }
```

```
1 str(res)
```

```
'data.frame':  20 obs. of  3 variables:
 $ Group   : Factor w/ 2 levels "Control","Treatment": 1 1 1 1 1 1 1 1 1 1 1 ...
 $ Subject: Factor w/ 20 levels "1","2","3","4",...: 1 2 3 4 5 6 7 8 9 10 ...
 $ RT      : num  0.535 0.486 0.509 0.516 0.51 ...
```

```
1 head(res, 3)
```

	Group	Subject	RT
1	Control	1	0.5354760
2	Control	2	0.4860800
3	Control	3	0.5091611

```
1 tail(res, 3)
```

	Group	Subject	RT
18	Treatment	18	0.3940291
19	Treatment	19	0.3983075
20	Treatment	20	0.4807047

```
1 mean(res$RT[res$Group == "Control"])
```

```
[1] 0.5142776
```

```
1 mean(res$RT[res$Group == "Treatment"])
```

```
[1] 0.4476666
```

Now let's practice!

Open this link and keep it open:

<https://etherpad.wikimedia.org/p/arca-corsoR>